Tabla de contenidos

Funciones en SQL Server....3
¿Qué son las funciones en el contexto de SQL Server?....3
Tipos de funciones en SQL Server....3
Categorías de funciones dentro del motor de base de datos....4
Escalares....4
Referidas a configuración....4
Agregación....4
Manejo de fecha y hora....5
Conversión de tipo de dato....5
Funciones matemáticas....6
Manejo de cadenas de caracteres....7
Gestión de cadenas de caracteres....7
Funciones tabulares en línea, concepto y características....9
Funciones tabulares Multi Sentencia....9
Concepto de variables en SQL Server....10
Declaración de variables....11
Procedimientos Almacenados de SQL Server....12
¿Qué son los procedimientos almacenados?....12
Características de los Stored Procedures....12
Principales ventajas del uso de Stored Procedures....12
Creación de un Stored Procedure....13
Ejecución de un Stored Procedure ya existente....14
Modificación de procedimientos almacenados....14
Eliminación de un Stored Procedure....14
Uso de parámetros de entrada y salida en procedimientos almacenados....15
Concepto de llamado por nombre y por posición....15
Gestión de errores y excepciones....15
Uso de las sentencias Try & Catch en SQL....15
Manejo de errores en SQL....16
Seguridad en SQL Server....17
Introducción....17
Tipos de autenticación en SQL Server....17
Estrategias utilizadas para fortalecer la seguridad en SQL Server....18
Usuario System Administrator (SA)....19
Usos típicos de la cuenta SA....19
Mejores prácticas en cuanto al uso de SQL Server....20
Riesgos reales del uso de la cuenta SA....20
Autenticación en SQL Server....20
Creación de un LOGIN....21
Creación de un usuario asociado a un LOGIN....21
Concepto de esquemas en Bases de Datos....22
Integridad de datos....22
Evaluación y herramientas de seguridad en SQL Server....22
Tipos de amenazas más comunes en SQL....23
Inyección SQL (SQL Injection)....23
Riesgos de canal lateral....23

Amenazas de infraestructura....24
Concepto de entidad de seguridad en SQL Server....24
Entidades de seguridad a nivel del motor de base de datos....24
Entidades de seguridad a nivel de base de datos....25
Concepto de usuario y esquema dbo....25
Rol público de servidor y de base de datos....25
INFORMATION_SCHEMA, usuarios y esquemas sys....25
Usuario guest....26
¿Que son los elementos protegibles?....26
Ámbito protegible de servidor....26
Ámbito protegible de base de datos....26
Ámbito protegible de esquema....27
Control de acceso a un elemento protegible....27
Cifrado y certificados en SQL Server....28
Consideraciones importantes acerca del uso de cifrado y certificados....28
Funciones de Transact-SQL....29
Certificados, concepto y características....29
Claves simétricas y asimétricas....29
Cifrado de datos transparente TDE....30
Transacciones y bloqueos....30
Introducción a niveles de aislamiento....30
Niveles de aislamiento....31
READ UNCOMMITED....31
READ COMMITED....31
REPEATABLE READ....31
SERIALIZABLE....31
Nuevo nivel de aislamiento de instantánea....31
SNAPSHOT Isolation Level....31
Características de transacciones implícitas....32
Modo por defecto en SQL Server....32
Variables tabla en SQL Server....32
Variables del sistema....33
Triggers....33
¿Qué son los Triggers?....33
Usos más comunes de los triggers....33
Consideraciones importantes en el uso de Triggers....34
Categorías de Triggers....34
Triggers DML....34
Triggers DDL....34
Creación de un procedimiento Trigger....35
Restricciones....35
Modificación y eliminación de un Trigger ya existente....35
Consultas dinámicas en SQL Server....35
Concepto de consultas dinámicas y características más relevantes....35
Cursores....36
¿Qué es un cursor en SQL Server?....36
Creación e instrucciones de un Cursor....36

Funciones en SQL Server

¿Qué son las funciones en el contexto de SQL Server?

Las funciones en SQL Server corresponden a un tipo de estructura en la cual se define un **conjunto de sentencias** que deben funcionar en grupo de manera lógica y organizada. De una manera similar a lo que sucede en lenguajes de programación tradicionales, al momento de definir una función, debemos asignarle un **nombre** para poder identificarla. Por otra parte, y de manera opcional, se pueden especificar **parámetros de entrada** o argumentos para la función. En cualquiera de éstos casos, una función retornará un valor que podrá ser usado para otras tareas o alimentar otros bloques de código.

Tipos de funciones en SQL Server

Existen dos tipos principales de funciones a las cuales se podrá acceder en SQL Server, las cuales son las **funciones definidas en el sistema** y las **funciones definidas por el usuario**. Pasamos a detallar de manera breve las características de ambas:

- Funciones definidas en el sistema → son aquellas funciones que vienen **incorporadas** dentro del motor de la base de datos. No requieren intervención alguna por parte del usuario para ser definidas o implementadas porque se encuentran **pre compiladas** dentro del motor de la BD. Ejemplos de este tipo de funciones podrían ser **GETDATE()** para retornar la **fecha actual** del motor, **GROUP BY** para realizar **agrupaciones** en resultados de consultas, etc.
- Funciones definidas por el usuario → las funciones definidas por el usuario se caracterizan por ser enteramente implementadas por el usuario del motor. Es él quien determina el nombre de la función, si recibirá parámetros de entrada (y en caso que así sea, el nombre y tipo de dato de los mismos). Permitirá entonces que ciertas tareas que no puedan ser cubiertas con funciones del sistema (o bien que puedan ser realizadas de manera más **eficiente**), se lleven a cabo a través de este tipo de función.

Debido a la gran variedad de funciones incluidas dentro del motor de base de datos, a continuación pasamos a abordar las distintas categorías que podemos encontrar en cuanto a cuales son las funciones más utilizadas y su utilidad dentro de SQL Server.

Categorías de funciones dentro del motor de base de datos

Una vez comprendido el concepto de qué es una función del sistema, se presentan las distintas categorías de funciones. Se dividen según la utilidad o el contexto en el cual se va a hacer uso de ellas.

Por otra parte, es necesario además, tener en cuenta que debido a la naturaleza de **inmutabilidad** de estas funciones, no es posible modificar las distintas **restricciones de uso** que pueden tener, y, que en ciertos contextos o consultas, existirán funciones que **no podrán** ser utilizadas en conjunto.

Escalares

Se le llama escalares a aquellas funciones que se caracterizan por tomar como **parámetro de entrada un único valor**, además que como resultado o retorno también devuelven un **único valor**.

Las funciones escalares se utilizan típicamente en el contexto de consultas de tipo **SELECT**, **WHERE**, **ORDER BY**, etc.

Referidas a configuración

Las funciones de configuración se utilizan principalmente para obtener **valores informativos** sobre el estado actual del sistema donde corre la instancia de SQL Server. Algunos ejemplos de ésto podrían ser:

- La versión actual de SQL Server
- La base de datos actual en la que se está trabajando
- El nombre del equipo conectado
- El nombre del usuario conectado

Sin embargo, existen muchas más funciones que no se describen en este párrafo.

Agregación

Hablamos de funciones de agregación o “de agregado” para aquellas que son utilizadas en el contexto de consultas a la base de datos, a fin de realizar un **cálculo** sobre un conjunto de valores obtenido por dicha consulta. Como valor de retorno, van a devolver un **único valor**.

Algunos ejemplos conocidos de este tipo de funciones que se pueden encontrar a menudo al trabajar con SQL Server son las funciones **COUNT()**, **SUM()**, **MIN()**, **MAX()**, entre otras tantas.

Manejo de fecha y hora

Dentro de esta categoría de función, se pueden encontrar dos tipos: funciones para **obtener información de fecha y hora**, así como funciones que permitan realizar distintos tipos de **cálculos con fechas**. Esto a fin de evitar problemas de precisión si se intentara realizar los cálculos manualmente.

Ejemplos de ellas son: **GETDATE()**, la cual permitirá al usuario obtener la **fecha y hora actual del servidor** en el cual se está trabajando; así como también **SYSDATETIME()** que devuelve la fecha y hora con una **precisión mayor** que la función anterior.

Por otra parte, con respecto a las funciones para operar con fechas, se mencionan a continuación las dos más comunes y de mayor utilización, junto con ejemplos de cada una de ellas:

DATEADD() → se utiliza para poder sumar o restar un **intervalo** a una fecha dada. Se la puede encontrar expresada de una forma similar a los siguientes ejemplos:

- Sumar un día a una fecha determinada → SELECT DATEADD(day, 1, GETDATE())
- Sumar un mes a una fecha determinada → SELECT DATEADD(month, 1, GETDATE())
- Restar un año a una fecha determinada (sumando -1) → SELECT DATEADD(year, -1, GETDATE())

DATEDIFF() → en contraste con el caso anterior, esta función se encarga de obtener la **diferencia entre dos fechas** dadas, tomando en cuenta el periodo de tiempo que el usuario desee calcular, sean días, meses o años. A continuación se muestra un ejemplo de cómo se debe usar para cada tipo de intervalo:

- Días de diferencia entre dos fechas → SELECT DATEDIFF(day, ‘2020-01-01’, GETDATE())
- Meses de diferencia entre dos fechas → SELECT DATEDIFF(month, ‘2025-01-01’, GETDATE())
- Años de diferencia entre dos fechas → SELECT DATEDIFF(year, ‘2025-01-01’, GETDATE())

Conversión de tipo de dato

Como su nombre lo indica, las funciones de conversión de tipo de dato permiten modificar correctamente el tipo de dato de una variable o valor para que se ajuste a la necesidad del usuario, como por ejemplo convertir un valor **DATETIME** (una fecha) en un tipo **VARCHAR** (cadena de caracteres).

Para poder hacer uso de la conversión de datos debemos llamar a la función **CONVERT()**, se muestra un ejemplo de cómo realizar la operación anteriormente descripta :

- SELECT CONVERT(VARCHAR(10), GETDATE(), 103)

Dado el ejemplo anterior, se puede apreciar que en este caso la función CONVERT() toma **tres argumentos**, el primero de ellos es el **tipo de dato final**, luego le sigue el **valor a convertir**, en este caso se llama directamente a la función GETDATE() la cual devuelve un **DATETIME**, y por último el valor 103 hace referencia al **formato de fecha** que se quiere utilizar, en este caso, **dd/mm/aaaa**.

Para continuar con las funciones de conversión, existe otra opción también muy utilizada la cual es la función **CAST()**, la cual se usará para realizar "casteos", es decir, para pasar rápidamente de un tipo de datos a otro.

- DECLARE @dato VARCHAR(2), @dato2 INT
- SET @dato = '10'
- SELECT @dato + '2'
- SET @dato2 = CAST(@dato AS INT)
- SELECT @dato2 + '2'

En el ejemplo anterior se puede ver que se han declarado **dos variables**, la primera de ellas de tipo **VARCHAR** y longitud 2; mientras que la 2da de ellas es de tipo Integer.

Se asigna el valor '10' a la variable **@dato**, y luego se ejecuta el SELECT de esa variable junto a '2'. Se puede ver, dadas las **comillas simples**, que en todo momento se trabaja con cadenas de caracteres y no con valores numéricos.

Y a continuación se realiza el casteo, dado que mediante la función **CAST()** se asigna a la variable **@dato2** un valor resultado de **castear @dato como INT**.

Funciones matemáticas

Esta categoría engloba a aquellas funciones usadas para realizar de manera eficiente cálculos matemáticos de variada índole como por ejemplo obtener un valor absoluto o redondear números con decimales.

- SELECT ABS(-55) → se obtiene el módulo de un número (el valor absoluto, es decir, sin signo).
- SELECT ROUND(120.4967, 2) → dado un valor, se especifica la cantidad de decimales necesaria.

Manejo de cadenas de caracteres

Dentro de esta subcategoría se encuentran una serie de funciones pre implementadas por SQL Server que revisten gran importancia debido a la versatilidad que ofrecen al trabajar con cadenas de texto. Se pasa a detallar de manera breve las más conocidas.

SUBSTRING(cadena, inicio, longitud) → usada para obtener una cadena a partir de otra. Se toma como valor de entrada la cadena de texto de la cual se va a extraer una subcadena, luego el inicio que será el índice del carácter desde el cual empezará la subcadena, y por último la longitud en caracteres que deberá tener la subcadena resultante.

- SELECT SUBSTRING('Hello World', 1, 5); → devolverá "Hello"

CHARINDEX(subcadena, cadena, inicio) → en este caso implica realizar una búsqueda de una subcadena a partir de otra dada usando como valor de referencia la **ubicación especificada como inicio**. La posición inicial siempre se cuenta a partir del **índice 1**, nunca en cero. Dado que en caso de retornar 0, indica que la subcadena especificada **no fue encontrada**.

- SELECT CHARINDEX('OM', 'CUSTOMER') → devuelve 5

PATINDEX(%patrón%, cadena) → básicamente será utilizado para obtener patrones determinados dentro de una cadena, es decir, dada una cadena de caracteres, retornará el **índice de inicio** del patrón buscado, y en caso de no encontrarlo retorna **0 (cero)**.

- SELECT PATINDEX(%ell%, 'Hello World'); → retornará como resultado 2

Gestión de cadenas de caracteres

Dentro de este apartado se hace referencia a las funciones de SQL Server para gestionar cadenas caracteres, permitiendo al usuario tareas como convertir a **valores numéricos** una cadena dada, obtener su **longitud**, extraer **valores ASCII**, entre otras.

LEN(cadena) → tomando como único valor de entrada la cadena que se quiere analizar, retornará un valor entero correspondiente a la **cantidad de caracteres que componen la cadena**.

- SELECT LEN('Hello') → retornará como resultado 5

CHAR(carácter) → toma como argumento **un único carácter**, retornará su valor ASCII correspondiente.

- SELECT CHAR(65) → retornará como resultado 'A'

RIGHT(cadena, longitud) → esta función acepta como valores de entrada una cadena y un valor entero. Se encarga de leer desde la derecha hasta el **índice indicado** por el **valor ingresado**, y retorna como resultado la **cadena obtenida** leída hasta ese punto.

- SELECT RIGHT('Hello World', 3) → retornará como resultado 'rld'

LEFT(cadena, longitud) → sigue un proceso prácticamente idéntico al caso anterior, solo que comienza a leer desde la **izquierda**.

- SELECT LEFT('Hello World', 5) → retornará como resultado 'Hello'

LOWER(cadena) → como su nombre indica, la cadena ingresada como parámetro se retorna en **minúsculas**.

- SELECT LOWER('HELLO WORLD') → retornará como resultado 'hello world'

UPPER(cadena) → idéntica a la anterior, se encarga de retornar la cadena ingresada en **mayúsculas**.

- SELECT UPPER('hello world') → retornará como resultado 'HELLO WORLD'

LTRIM(cadena) → permite eliminar los espacios en blanco a la **izquierda** de la cadena.

- SELECT LTRIM(' hello') → retornará como resultado 'hello'

RTRIM(cadena) → ídem al elemento anterior, solo que elimina los espacios a partir de la derecha.

- SELECT RTRIM('hello ') → retornará como resultado 'hello'

REPLACE(cadena, cadena que se va a ingresar, cadena que se reemplaza) → esta función reviste una gran importancia y utilidad ya que va a permitir reemplazar distintas subcadenas por otras dentro de una determinada. Así por ejemplo, pueden formatearse datos como direcciones URL, emails, etc. Toma como parámetros tres valores en el siguiente orden: la cadena sobre la cual se va a trabajar, luego seguido de la cadena que se quiere **ingresar**, y por último la cadena que será **reemplazada**.

- SELECT REPLACE('zzz.unlam.edu.ar', 'z', 'w') → retornará como resultado 'www.unlam.edu.ar', debido a que todas las ocurrencias de la cadena **'z'** se sustituyen por **'w'**.

REPLICATE(cadena, numero de veces) → tomando como valores de entrada una cadena y un valor entero positivo, **repite** la cadena indicada la cantidad de veces que especifica el parámetro.

- SELECT REPLICATE('hello', 3) → retornará como resultado 'hellohellohello'

SPACE(cantidad) → toma como único parámetro un valor entero positivo que hace referencia a una **cantidad de espacios** que el usuario quiere **insertar**. Permite entonces concatenar cadenas de texto de manera fluida y natural al poder realizar sumas entre cadenas usando el operador **+** y la función insertará los espacios.

- SELECT 'Hola' + SPACE(1) + 'que tal' → retornará como resultado 'Hola que tal'

REVERSE(cadena) → función muy sencilla de usar, recibe como valor de entrada una cadena y la retorna **invertida**.

- SELECT REVERSE('hello') → retornará como resultado 'olleh'

Funciones tabulares en línea, concepto y características

Son un tipo de función muy útil en SQL que permiten lograr que una función retorne como tipo de dato un objeto **TABLE**, es decir, la función al ejecutarse retornará un **ROWSET** conformado por el **conjunto de resultados** de una consulta **SELECT** realizada durante la ejecución de la función en si misma. Se puede notar también, que **no requiere** ubicarse dentro de un bloque **BEGIN-END**, así como también puede **recibir parámetros** como cualquier otro tipo de función.

Se usa a menudo como forma de sustituir una **vista parametrizada**; incluso, pudiendo usar este valor dentro de sentencias JOIN, querys, etc; como si de una tabla tradicional se tratase.

A continuación se muestra un ejemplo sencillo de una función tabular en código SQL:

- CREATE FUNCTION miFuncionTabular (@parametro INT) -- se define el nombre y parámetro
- RETURNS TABLE -- acá se define el tipo de dato de retorno como **TABLE**
- AS
- RETURN -- el retorno como tal se compone del bloque siguiente entre paréntesis
- (
- SELECT …FROM… -- el bloque de sentencias ubicado aquí devuelve el **ROWSET**
-)

Funciones tabulares Multi Sentencia

Se conoce con este nombre a una combinación entre un **procedimiento almacenado** y una **vista**. Se caracteriza por contener **lógica compleja** y **múltiples sentencias** en su interior, que se aplican para construir la tabla que conformará el dato retornado por la función. Un ejemplo de uso típico de una función tabular multi sentencia es en la **cláusula FROM** al realizar un **SELECT**.

Una diferencia sustancial con las funciones tabulares en línea son que, dentro de su definición, el grupo de sentencias se encontrará **dentro** de un **bloque BEGIN-END**.

Se muestra un ejemplo de cómo se vería una función tabular multi sentencia:

- CREATE FUNCTION miFucionTabularMultiSentencia (@parametro NVARCHAR(5))
- RETURNS @tablaResultado TABLE -- retorna un objeto de tipo TABLE

- (campo1 INT PRIMARY KEY, campo2 NVARCHAR(50)) -- se declara la tabla y sus campos
- AS
- BEGIN -- tal como se explico antes, las función se encierra con BEGIN-END como un SP
- -- dentro de este bloque se definirán las consultas necesarias
- -- para completar los parámetros de la tabla que compone el
- -- dato de retorno
- RETURN -- se retorna la variable en esta línea (el contenido de la tabla creada)
- END;

Al analizar ambos ejemplos, se aprecia la diferencia tanto de **robustez** como de **lógica anidada** que existe entre las funciones tabulares en línea y las multi sentencia.

Concepto de variables en SQL Server

Para abordar el tema de mejor manera, se puede comenzar analizando a qué hace referencia el concepto de **variable**. ¿Que se entiende por variable? De manera similar a lo que ocurre al trabajar con lenguajes de programación tradicionales, con T-SQL es posible definir variables que funcionen como contenedores o "cajas" con datos, los cuales están **sujetos a cambios** en caso que sea necesario. Por ejemplo, si se debe trabajar con una fecha dada, es posible guardarla en una variable, para que al momento de requerirla en una consulta su valor se **recupere directamente** desde esa variable y no sea necesario **recalcularla**.

Una vez aclarado el concepto general de variable, y ya adentrados en el contexto de las variables específicas en SQL Server, es posible destacar las siguientes características básicas de ellas:

- Son parte **específica de cada conexión** a la base de datos, por lo tanto al momento de desconectarse se **liberan**
- Para declararlas, se hace anteponiendo un **@** seguido del **nombre** que se le va a asignar a esa variable, el cual puede contener **cualquier carácter**. Si es importante recordar que los nombres de variables **no pueden contener espacios en blanco**

Declaración de variables

En cuanto a la declaración de variables, es importante tener en cuenta que es un requisito **obligatorio** para poder hacer uso de ellas (no se permitirá usar variables no declaradas). Por otra parte, dadas las restricciones de SQL Server, no es posible crear variables de tipo de dato **IMAGE**, **TEXT** o **NTEXT**. A continuación se muestra un ejemplo breve de cómo se ve una declaración de una variable de tipo numérico:

- DECLARE @variable INT;

Se puede apreciar otro detalle de vital importancia al momento de trabajar con una variable: **el tipo de dato**. Es muy importante seleccionar correctamente el tipo de dato acorde a la tarea que se debe realizar, dado que ciertos tipos pueden tener mayor precisión que otros, como por ejemplo un decimal comparado a un entero o bien un tipo DATETIME (fecha y hora) con DATE (solo fecha).

Por otra parte, otra característica que se puede apreciar en la línea de ejemplo anterior es que no se ha **inicializado** la variable, es decir, no tiene un valor ya asignado por el usuario. En estos casos, la variable tiene un valor asignado por SQL Server de **null**.

Entonces, una vez entendido cómo declarar una variable y cual es su valor por defecto sin inicializar, lo siguiente es saber cómo **asignarle un valor propio**. Para ello, es necesario usar la sentencia SET seguida de la variable en cuestión y luego el contenido que deberá tener, a continuación se muestra un ejemplo de cómo asignar la variable mostrada antes:

- SET @variable = 100;

Pero el ejemplo anterior no es la única manera de asignar contenido a una variable, ya que también se le puede asignar el **resultado** de una consulta como tal; por ejemplo, al intentar traer un nombre desde una tabla, podemos estructurar el SELECT de manera tal que cuando SQL Server retorne ese nombre en la consulta, automáticamente lo haga asignando ese valor a nuestra variable, de modo que se omite el uso de la sentencia SET en este caso. A continuación se muestra un ejemplo breve de como declarar una consulta de ésta índole:

- SELECT @variable = name FROM Production.Product WHERE ProductID = 999;

En el caso anterior, la consulta SELECT ha llamado a la tabla Production.Product en busca del registro con ID 999, y acto seguido aloja su nombre dentro de @variable.

Procedimientos Almacenados de SQL Server

¿Qué son los procedimientos almacenados?

Se conoce como procedimiento almacenado o **Stored Procedure** a un grupo de sentencias en T-SQL que se compilan dentro de un **plan de ejecución**.

Se utilizan como una herramienta para **optimizar tareas repetitivas** que suelen involucrar **variables definidas por el usuario** además de **cálculos** en muchos casos. O bien controlar el flujo de ejecución de una consulta y poder decidir si ejecutar ciertos bloques condicionales, etc. Se puede asociar con el concepto de **rutina**, ya que el fin de los procedimientos almacenados siempre será **encapsular código** y permitir a su vez que este último sea **reutilizable**.

Características de los Stored Procedures

Con el objetivo de comprender mejor cómo funcionan los procedimientos almacenados, la siguiente lista muestra una serie de características generales de estas estructuras:

- Pueden aceptar **parámetros de entrada** (no es obligatorio que los tengan)
- Devuelven siempre un valor escalar como **retorno**, el cual indica si la operación realizada por el Stored Procedure se llevó a cabo de manera exitosa o no
- Los Stored Procedures pueden **anidarse**, de modo que dentro del plan de ejecución es posible llamar a otros SPs
- Tienen la capacidad de poder retornar también valores en forma de **parámetros de salida**

Principales ventajas del uso de Stored Procedures

En la siguiente lista, se enumeran de manera breve las ventajas más importantes de las cuales se puede sacar mucho provecho al aplicar correctamente Stored Procedures en SQL Server:

- El código se vuelve **reutilizable** y **modular**, se encapsula mejor la **lógica de negocio**.
- El acceso a datos se hace mucho más **consistente** porque todas las aplicaciones conectadas a la base de datos pueden usar los **mismos procedimientos almacenados**.
- Se aumenta en gran medida la **seguridad** de toda la instancia del motor de base de datos, porque las tablas **no se exponen a las aplicaciones**, sino que los Stored Procedures hacen de intermediarios y **limitan el acceso** a la información sensible.

- Derivado del punto anterior, los Stored Procedures aceptan la definición de **permisos de acceso**, por lo tanto el usuario que no tiene acceso a una tabla puede igualmente usar el SP que usa esa tabla.
- El desempeño general de la instancia del servidor también se ve beneficiado, porque el plan de ejecución de los procedimientos se **establece y se compila solo la primera vez** que el procedimiento es utilizado, por lo tanto, en las siguientes invocaciones al SP directamente se reutiliza.
- Se disminuye el **uso de red** porque al usar un procedimiento almacenado, básicamente lo que sucede es que se le indica al servidor en **una sola sentencia** que realice una serie de tareas, sin requerir que esas consultas se envíen por separado. Por lo tanto en una sola invocación se obtiene un funcionamiento más complejo.
- Dado que los procedimientos almacenados se compilan con un set de instrucciones y parámetros específicos, se vuelven una herramienta muy útil para evitar **ataques de inyección SQL**.

Creación de un Stored Procedure

Una vez comprendido el concepto básico de qué es un Stored Procedure, lo propio es mostrar con un ejemplo de qué manera se estructura la sentencia necesaria para crear uno de ellos, la cual es la siguiente:

CREATE PROCEDURE nombre_procedimiento()

- AS
- BEGIN
- -- bloque de sentencias
- -- de plan de ejecución
- END
- GO;

En caso de especificarse la sentencia **WITH** al momento de crear el procedimiento almacenado, la misma puede ir acompañada de opciones como **RECOMPILE** (se forzará a recompilar el SP cada vez que sea utilizado porque **no se almacenará en caché**), **ENCRYPTION** (cifrar el contenido del SP para fortalecer la seguridad), entre otras.

Ejecución de un Stored Procedure ya existente

Una vez creado un procedimiento almacenado, para ejecutarlo bastará con hacer uso de la sentencia **EXECUTE** o en su defecto de su forma abreviada **EXEC**, seguida del nombre del procedimiento que se va a llevar a cabo.

- EXECUTE miProcedimientoAlmacenado

Cabe destacar, que en caso que el SP reciba parámetros de entrada en su invocación, se deben especificar luego del nombre en el orden declarado y respetando su tipo de dato (sin paréntesis).

Modificación de procedimientos almacenados

No solo es posible crear procedimientos almacenados en la instancia de SQL Server, sino que también se permite **modificarlos**, esto responde a las constantes necesidades y requerimientos que plantean los usuarios al momento de consumir información; o bien porque se realizan cambios en las **definiciones de las tablas**.

La ventaja que supone modificar un SP en lugar de volver a crearlo, será que se **mantienen los permisos asignados** al momento de su creación. Un detalle muy importante a tener en cuenta sobre este tema es que modificar un procedimiento almacenado solo afecta a ese procedimiento, es decir, si dentro de su plan de ejecución incluye invocaciones a otros SPs, éstos últimos **no se verán afectados por la modificación**.

Se muestra a continuación un ejemplo breve de cómo llevar a cabo la modificación:

- ALTER PROCEDURE miProcedimientoAlmacenado
- AS
- BEGIN
- -- Nuevas sentencias del plan de ejecución
- -- Sobrescriben al anterior plan
- END
- GO;

Eliminación de un Stored Procedure

En caso que un SP ya no sea necesario en el sistema, o que por cualquier motivo sea imperativo eliminarlo de la instancia, esta acción se llevará a cabo usando la sentencia **DROP** especificando el nombre del procedimiento que deberá ser eliminado.

- DROP PROCEDURE miProcedimientoAlmacenado

Uso de parámetros de entrada y salida en procedimientos almacenados

Es de vital importancia la correcta utilización de parámetros de tanto de **entrada** como de **salida** para aprovechar la flexibilidad de trabajo que ofrecen los Stored Procedures.

Al momento de la definición de un procedimiento, se declaran también los parámetros que **recibirá** o que **retornará** luego de la invocación. Se señalan con el signo @ seguido del nombre del parámetro dentro del contexto del SP, es decir, cuando el SP se ejecute, referirá con ese nombre al parámetro. A su vez, se declarará también el **tipo de dato**.

Por otra parte, para el caso de los **parámetros de salida**, la declaración será prácticamente idéntica, solo agregando la sentencia **OUTPUT** luego del parámetro para indicar que se trata de una variable que deberá ser retornada al final de la ejecución.

Concepto de llamado por nombre y por posición

Dado un procedimiento almacenado que recibe parámetros de entrada en su invocación, existen dos maneras de llamarlo:

- **Llamado por nombre**: se especifica el nombre del parámetro seguido del valor que se le quiere dar para ejecutar el SP, de esta forma, el orden de los parámetros **no afectará a la ejecución** ya que se referenciará directamente por nombre de la variable.
- **Llamado por orden**: es lo que se entiende como **pasaje por posición**, en este caso, solo se llamará al SP y se le darán los valores que deberá tomar para sus variables. Dado que se **respeta** el orden de variables utilizado al momento de crear el SP, se **infiere** por el orden de valores cual de ellos corresponde a cada parámetro dentro del plan de ejecución del SP.

Gestión de errores y excepciones

Uso de las sentencias Try & Catch en SQL

De una manera similar a lo que ocurre con un lenguaje de programación tradicional como por ejemplo Java o C#, existe la posibilidad que, dado un cierto grupo de instrucciones o sentencias, se

genere un error que deba ser apropiadamente gestionado por el motor de base de datos. Es para este tipo de situaciones que se aplica lo que se conoce como bloques **Try & Catch**. Este tipo de estructuras permite ejecutar de manera controlada un grupo de sentencias que podrían generar una excepción o error durante el proceso. En caso que ésto último suceda, contarán con definiciones para poder **revertir los cambios** en caso de ser necesario.

El flujo de trabajo se estructura de la siguiente manera:

- BEGIN TRY
- -- bloque de sentencias
- -- que pueden generar
- -- una excepción
- END TRY
- BEGIN CATCH
- -- sentencias de manejo
- -- de posibles errores
- END CATCH;

Se puede apreciar en el ejemplo anterior como el bloque se divide en dos partes, dentro del bloque Try se ejecutarán las instrucciones; que en caso de fallar **arrojarán** una excepción. Es en este momento que el flujo se traslada al bloque Catch, que **captura** la excepción y decidirá qué se debe hacer llegado ese punto, como por ejemplo **revertir** cualquier cambio no deseado.

Manejo de errores en SQL

SQL Server proporciona el siguiente grupo de sentencias a fin de tener la mayor cantidad de información posible en pos de gestionar posibles errores durante un procedimiento:

- **ERROR_NUMBER**: se obtiene el número de identificación interna del error
- **ERROR_STATE**: indica la fuente del error
- **ERROR_SEVERITY**: incluye información sobre un amplio abanico de cuestiones, desde errores informativos hasta errores que deban ser corregidos por el usuario DBA
- **ERROR_LINE**: la linea donde se ha producido el error
- **ERROR_PROCEDURE**: indicará el nombre del SP o la función

- **ERROR_MESSAGE**: de gran importancia debido a que devuelve información esencial y el mensaje de error en cuestión

Seguridad en SQL Server

Introducción

En el contexto de trabajo de una instancia de SQL Server productiva, es de vital importancia conocer las medidas de seguridad que se deben implementar para garantizar la confidencialidad y disponibilidad de la información. Un servidor de base de datos puede gestionar información de cientos o miles de usuarios, incluyendo nombres y apellidos, direcciones, números de identificación, teléfonos, sueldos, etc. Por lo tanto, los administradores de base de datos trabajan constantemente para conseguir la mayor robustez y seguridad posibles. Esto incluye limitar **quienes** acceden al sistema, no solo en cuanto a los usuarios externos que solo consumen la instancia sino también al **personal de la empresa**, limitando acceso solo a áreas pertinentes, usando vistas o SPs en lugar de acceder a tablas de manera directa, entre otras.

Tipos de autenticación en SQL Server

La siguiente lista expone los tipos de autenticación más usados en instancias de base de datos:

- Inicio de sesión con Windows, mediante cuenta de usuario local o mediante una cuenta de dominio **Active Directory** (entorno empresarial)
- Grupos de Windows, similar al caso anterior, se usa la autenticación mediante la cuenta de un equipo Windows, pero se estructuran en **grupos** de manera tal que todos aquellos usuarios de un grupo obtienen **permiso de acceso** al pertenecer al grupo autorizado
- Inicio de sesión de SQL, con este tipo de autenticación se guarda el nombre de usuario especificado junto con el valor del **hash** de su contraseña correspondiente; dentro de la base de datos **master**.
- Inicio mediante usuarios de base de datos independientes, que autentifican las conexiones a SQL Server directamente a **nivel de la base de datos a utilizar**. Esta base se caracteriza por estar **aislada** del resto de bases, así como también de la instancia.

Estrategias utilizadas para fortalecer la seguridad en SQL Server

La estrategia principal en cuanto a la seguridad al momento de trabajar con SQL se base en un criterio de **privilegios mínimos necesarios**, es decir, un usuario dentro de la instancia solamente puede tener los privilegios mínimos que le permitan llevar a cabo la acción por la cual acude a conectarse o consumir información de la base de datos. Por ejemplo, imaginando el caso de una empresa con distintas áreas, un usuario del área de **Marketing** no debería tener privilegios elevados como un **sysadmin** o alguien asignado al área de **Seguridad Informática**. Esto es así debido a que en caso de verse comprometido este usuario, afectaría al resto de la organización al poder acceder a información sensible o confidencial. Por lo tanto, siempre se busca limitar los **roles de usuario** a lo mínimo necesario para garantizar el funcionamiento correcto del sistema.

Y basado en lo comentado antes, es que cobra tanta importancia el uso de Active Directory, ya que es muy común ubicar los usuarios en **grupos** con permisos determinados y roles puntuales dentro de la base de datos. Por otra parte, también se facilita en gran medida remover usuarios si por ejemplo **abandonan** la empresa o cuando sea necesario **modificar** sus roles.

Otro punto muy importante a tener en cuenta, y de uso extendido no solo en SQL Server sino en general, es el sistema de **MFA** o Autenticación Multifactor. Permitirá evitar el robo de credenciales ya que incluso si un atacante logra vulnerar la seguridad y hacerse con credenciales de usuario con privilegios elevados, la autenticación multifactor obliga a usar un **método extra de validación** al iniciar sesión como podría ser un código enviado a la casilla de **email** del usuario o un **SMS** a su teléfono, debiendo tener en este último caso acceso **físico** al equipo del usuario.

Una estrategia muy extendida, y que en gran medida se aplica en sistemas con bases de datos, es el uso de contraseñas **robustas** y **complejas**, dado que cuanto más extensa y compleja sea, más difícil será adivinarla. Igualmente, cabe recordar que incluso aunque las contraseñas utilizadas sean consideradas seguras, es una muy buena práctica llevar a cabo **actualizaciones** de las mismas cada cierto tiempo.

En este punto, es propicio mencionar que mediante **cuentas de servicio administradas en grupo**, es posible gestionar de manera automatizada y rápida las distintas contraseñas; así como también simplificar cómo se administran los **nombres de entidad de seguridad**, pudiendo delegar responsabilidad de administración en varios agentes.

Por otra parte, es muy importante mencionar en cuanto a la seguridad es la minimización de derechos concedidos a la cuenta de Active Directory DBA (Database Admin). Es decir, se busca activamente **separar tareas** de modo que se limite el acceso a la VM, la capacidad o alcance para crear nuevos inicios de sesión en el SO se reduce, así como también la capacidad de modificar registros de errores o instalar aplicaciones y características.

Por último, una muy buena práctica es revocar el rol de **sysadmin** de las cuentas DBA y en reemplazo otorgar el rol de CONTROL SERVER; debido a que un usuario con rol de **sa** no respeta **DENY**, en cambio CONTROL SERVER se ve **obligado** a hacerlo.

Usuario System Administrator (SA)

Comúnmente conocida como cuenta **sa**, se trata del usuario predeterminado al momento de realizar la instalación de SQL Server. Se caracteriza por ser una cuenta con **privilegios y permisos muy elevados**, la cual se utiliza principalmente para tareas administrativas.

Dados sus elevados privilegios, permite realizar cualquier tipo de acción dentro de la instancia del servidor, y es ésta característica la cual determina que deba ser usada lo **mínimo posible**, y solo por parte de **personal autorizado** y en situaciones críticas que no puedan ser resueltas de otra manera. Esto es así porque en entornos de producción reales, se busca usar cuentas con permisos **limitados** solo a las tareas necesarias.

En la siguiente lista, se enumeran características relevantes acerca del rol **sysadmin** acompañadas de una breve explicación:

- **Privilegios asignados**: posee asignados **permisos totales** a todas las bases de datos y objetos existentes en el servidor sin importar las configuraciones de tablas, vistas y procedimientos.
- **Autenticación**: se aplica el tipo de **autenticación de SQL Server**, se usa un nombre de usuario y una contraseña definida.
- **Rol asociado**: la cuenta **sa** se caracteriza por tener un **rol fijo**
- **Seguridad**: durante la instalación de SQL Server, se puede elegir si habilitar o deshabilitar el usuario SA. En caso de ser habilitado, es imperativo asignar una contraseña lo más robusta posible.

Usos típicos de la cuenta SA

En la lista presentada a continuación se detallan las principales tareas de administración y mantenimiento del servidor que pueden llevarse a cabo mediante la cuenta **sa**.

- **Administración**: incluye acciones como crear y eliminar bases de datos dentro de la instancia, modificar sus configuraciones, establecer los permisos y roles de otros usuarios, así como también ejecutar tareas críticas de mantemiento.
- **Emergencias**: en caso que otros usuarios con privilegios elevados pierdan acceso al servidor, la cuenta **sa** puede servir como una cuenta de respaldo.

- **Pruebas y desarrollo**: debido al gran nivel de acceso, en ciertas circunstancias puede acelerar ciertas tareas de desarrollo, aunque se considera una práctica no recomendable.

Mejores prácticas en cuanto al uso de SQL Server

Se pueden notar como mejores prácticas en cuanto al uso de la cuenta **sa** en SQL Server a las siguientes:

- De no ser necesario, la cuenta **sa** debería permanecer **deshabilitada**
- En caso de usarse autenticación de Windows teniendo ya registrados otros administradores, también se debe **desactivar** la cuenta **sa**
- Si se vuelve imposible evitar el uso de la cuenta **sa**, no se recomienda mantener la contraseña que se crea al momento de instalar la instancia en el sistema
- Restrinir el acceso directo a la cuenta
- Usar **restricciones de acceso** mediante mapeo de direcciones IP puntuales, es decir, solo permitir conexiones entrantes desde IPs verificadas
- **Monitorear** de manera constante el uso de la cuenta y activar **registros** o **logs de eventos**.

Riesgos reales del uso de la cuenta SA

Los riesgos que involucra el mal uso de la cuenta **sa** incluyen la **vulnerabilidad** ante ataques informáticos ya que un atacante que logre hacerse con las credenciales de la cuenta SA tendrá toda la instancia del servidor a disposición. E incluso aunque no se trate de un ataque externo, no se debe dejar de tener en cuenta un posible **error humano** que pueda causar daño significativo.

Autenticación en SQL Server

Al momento de realizar el ingreso a la instancia del motor de base de datos, SQL Server presenta dos modos de autenticación los cuales son la autenticación mediante **usuario de Windows** y luego con **credenciales de SQL** (conocido también como **autenticación mixta**). Siempre se recomienda el primer método ya que se aplican las mismas restricciones de acceso del sistema operativo por lo tanto genera una **capa extra** de seguridad. Además, este tipo de autenticación va a permitir otorgar acceso tanto a **usuarios** como a **grupos** del sistema.

Por otra parte, la autenticación mixta con credenciales de SQL solo se usará en casos en los cuales la infraestructura de red en la que se ubique el equipo que sustenta la instancia **no permita usar** solo la autenticación de Windows.

Creación de un LOGIN

Dentro de la instancia del servidor, el proceso de acceso a la base de datos consta de dos pasos: el primero de ellos consiste en crear la **conexión** a la base de datos, es decir, a nivel de la instancia de SQL Server se crear lo que se conoce como el **LOGIN** (el cual contiene las definiciones de nombre de usuario y contraseña), actúando éste último como la primer puerta de entrada a la BD. Luego, en caso que ese usuario cuente con los permisos correspondientes para una base puntual, podrá efectivamente realizar operaciones.

A fin de hacer más completa la explicación, se muestra un ejemplo en código SQL que permite crear un LOGIN o inicio de sesión de Windows:

- CREATE LOGIN [EXPRESS\nombre_usuario] FROM WINDOWS;

En este caso, la sentencia **EXPRESS** refiere al dominio o bien al equipo en el cual se encuentra instalado SQL Server; mientras que **nombre_usuario** hace referencia al usuario del sistema operativo. Una vez ejecutada la sentencia; y por supuesto en caso que el usuario exista en el sistema, queda registrado para acceder a la instancia.

Por otra parte, es posible crear un LOGIN de SQL especificando tanto el nombre asociado al inicio de sesión como la contraseña que usará para conectarse. Se muestra un ejemplo a continuación:

- CREATE LOGIN username WITH PASSWORD = ‘password’;

Creación de un usuario asociado a un LOGIN

Dado un inicio de sesión (el **LOGIN**) creado anteriormente, para poder acceder a una base de datos específica, se debe asociar el inicio de sesión a un usuario **creado en la BD**. Es decir, el LOGIN permitirá la conexión a la BD, y asociarlo a un usuario permitirá usar esas credenciales para operar. En el ejemplo a continuación se muestra cómo se debe estructurar la sentencia en SQL:

- USE db;
- GO;
- CREATE USER User FOR LOGIN[Express\Usuario];

Concepto de esquemas en Bases de Datos

Los esquemas forman parte en SQL Server desde el año 2005, en el cual fueron introducidos por primera vez, y prevalecen hasta la fecha como una herramienta para **agrupar** objetos dentro de la base de datos siguiendo un **criterio** adecuado. Esto incluye los objetos de BD, tablas, vistas, SPs, etc. Y dada esa relación de pertenencia, el nombre del esquema formará parte también del nombre de los objetos pertenecientes a tal esquema. Por ejemplo, en caso de tener un esquema llamado **Ventas**, para crear una tabla que queremos llamar **Pedidos**, el nombre que deberá tener será **Ventas.Pedidos**. Pero la gran ventaja que ofrece el uso de esquemas es que permiten la **asignación de permisos** dentro de ese esquema, de manera tal que el acceso a los objetos no deberá ser configurado de manera manual en cada uno de ellos sino que el nivel de permiso asignado al esquema afecta a **todos los objetos dentro del mismo**.

En la siguiente linea se muestra un ejemplo de sentencia de creación de esquema en SQL:

- CREATE SCHEMA Ventas;
- GO;
- -- creación de la tabla dentro del esquemas
- CREATE TABLE Ventas.Pedidos(...);

Integridad de datos

Este concepto hace referencia a mantener un **registro histórico** de cambios en los datos guardados en la BD. Esto sería útil por ejemplo en situaciones críticas en las cuales se deban revertir cambios no deseados en el sistema, para realizar **auditorías** de cambios de aplicación o recuperar elementos eliminados por un **agente malintencionado**.

Evaluación y herramientas de seguridad en SQL Server

En esta sección, se aborda en detalle las herramientas que se deben aplicar para evaluar el nivel de seguridad efectiva en la instancia y de qué manera mejorarla.

Lista de procedimientos:

- **Configuración de área de superficie**: cuántas menos características se **activen** dentro del servidor, menos chances tendrá un posible atacante para intentar vulnerar el sistema.
- **Evaluación de vulnerabilidades mediante SQL Server Management Studio**: es una herramienta integrada dentro del IDE oficial de Microsoft que permite **detectar** y **corregir** vulnerabilidades en la base de datos.

- **Detección y clasificación de datos de SQL**: los **usuarios DBA** deben ser conscientes del tipo o sensibilidad de los datos con los cuales trabajan, tratando con la **confidencialidad correspondiente** aquellos datos que así deban ser gestionados.

Tipos de amenazas más comunes en SQL

Es de vital importancia para un administrador de base de datos conocer qué clase de amenazas se le presentarán al momento de gestionar una instancia de base de datos. Solo así será capaz de comprender cuáles son los puntos del servidor que más se podrían ver afectados; y además tomar medidas de prevención con el tiempo y complejidad suficientes para evitar daños al sistema.

Inyección SQL (SQL Injection)

El ataque de **inyección SQL** consiste en insertar código SQL malicioso dentro de cadenas de texto que sean posteriormente procesadas por una instancia de SQL Server para ejecutarlas. Básicamente, se incluye **dentro de la cadena** una serie de instrucciones diseñadas para explotar una vulnerabilidad aprovechando que el servidor ejecutará cualquier consulta **válida sintácticamente** dentro de tal cadena, dejando al descubierto información que de otro modo no debería ser accesible desde fuera.

Dado el tipo de amenaza que representa un ataque de inyección SQL, las medidas de prevención que mejor pueden mitigar tal situación son: **auditar** cualquier proceso SQL que pueda construir consultas con chances de destapar vulnerabilidades o información sensible, usar **instrucciones SQL parametrizadas**, revisar cualquier consulta que involucre realizar sentencias **EXEC** o **EXECUTE**, o incluso **limitar** el ingreso de caracteres que sean usados en consultas como por ejemplo delimitadores de cadenas, comentarios, procedimientos, etc. Por otra parte, y muy ligado con el ítem anterior, cualquier entrada que realice un usuario debe ser **validada** así como **limpiar** las **salidas de error** que pueden brindar información a un atacante.

Riesgos de canal lateral

Las medidas para mitigar los riesgos de canal lateral incluyen los siguientes procedimientos:

- Usar las versiones de software y aplicaciones más **recientes** en todo momento
- Aplicar los parches necesarios al **firmware** para todo el **hardware** de procesamiento
- En caso de trabajar con **Azure**, las cargas de trabajo y aplicaciones confidenciales deben correr en **máquinas virtuales aisladas**.

Amenazas de infraestructura

Existen una serie de amenazas concernientes a la infraestructura que pueden ser:

- **Acceso por fuerza bruta**: consiste en realizar reiterados intentos de inicio de sesión probando distintas contraseñas hasta adivinar la correcta y acceder al sistema.
- **Difusión de contraseña**: se busca vulnerar una cuenta con una contraseña diseñada específicamente para muchas cuentas. En caso que tal clave no resulte exitosa, se diseña una nueva también aplicada a muchas cuentas para intentar de nuevo el logueo; no sin antes esperar un tiempo prudencial para evitar que el sistema detecte actividad sospechosa. Medidas que pueden mitigar este riesgo son el uso de contraseñas **robustas y complejas**, así como también usar una cuenta de administrador que **no se llame** Administrador, es decir, usar una cuenta con los permisos de administrador necesarios que a simple vista **no parezca administrador**, a fin de confundir a posibles atacantes.
- **Ataques de ransomware**: consiste en provocar que la víctima del ataque descargue o permita la ejecución de **software malicioso** en el sistema, mediante un engaño al usuario para inducirlo a confiar en tal software. Una vez ejecutado, el software **cifra** o **encripta** todo el contenido del equipo y solicita una suma en dinero (generalmente en **criptomonedas**) a cambio de la clave necesaria para descifrar la información. Medidas de prevención que pueden mitigar esta clase de ataques son el uso de **firewalls**, actualizar el software utilizado y aplicaciones, usar cuentas pertenecientes a **grupos**, limitar el acceso a **máquinas virtuales**, evitar características o funciones que no sean necesarias (**desactivarlas**), etc.

Concepto de entidad de seguridad en SQL Server

Se conoce como entidad de seguridad a aquellos **individuos, grupos o procesos** que tienen acceso a SQL Server. Y a raíz de ésto, surgen lo que se conocen como **elementos protegibles**, los cuales son el servidor, la base de datos y aquellos objetos incluidos en la base de datos. Todos estos objetos cuentan con una serie de **permisos** los cuales se pueden modificar para aumentar o reducir su exposición dentro de SQL Server y en consecuencia qué tan seguros se los considera.

Entidades de seguridad a nivel del motor de base de datos

Este nombre hace referencia a las entidades con la capacidad de solicitarle recursos a SQL Server. Organizadas en jerarquías, su alcance de acción depende de cómo haya sido definida, es decir, que puede trabajar a nivel Windows, servidor o base de datos (orden descendente). Además, depende también si la entidad en cuestión es **indivisible o es una colección**. Por ejemplo, una entidad indivisible podría ser un inicio de sesión de Windows, en cambio, un grupo podría calificarse como una colección.

Ejemplos de entidades de seguridad a nivel servidor que se pueden nombrar son: Inicios de sesion de SQL, usuarios individuales y de grupos de Windows, Roles de servidor, etc.

Entidades de seguridad a nivel de base de datos

Por otra parte, a nivel de base de datos existen entidades de seguridad como pueden ser

- **Usuario de base de datos**
- **Rol de base de datos**
- **Rol de aplicación**

Concepto de usuario y esquema dbo

Se conoce como usuario **dbo** a una entidad especial presente en cada base de datos. Se caracteriza por tener **todos los permisos** dentro de la base a la cual pertenece, además de que no puede ser **eliminado** o **limitado**. Como tal, representa al **propietario** de la base pero no se debe confundir con el rol **db_owner** ya que no son lo mismo.

Por otra parte, se puede notar que el usuario **dbo** es propietario a su vez del esquema homónimo, predeterminado para todos los usuarios a menos que otro sea especificado. Igual a lo que sucede con el usuario, el esquema **dbo** no podrá ser **eliminado**.

Rol público de servidor y de base de datos

Se puede afirmar que cada inicio de sesión forma parte del rol de servidor **public**, así como también dentro de una base de datos puntual, cada usuario pertenece al rol de base de datos **public**. Incluso, si un usuario o una entidad de inicio de sesión no tiene asignados o denegados **permisos** concernientes a un objeto **protegible**, entonces **hereda** aquellos permisos del objeto **public**. Sin embargo, es importante saber que también podrá **revocar** permisos de roles **public**. La ventaja del uso de este rol es que posee los permisos típicos para la gran mayoría de tareas a realizar en la BD, por lo tanto cualquier operación de rutina se podrá llevar a cabo con este rol.

INFORMATION_SCHEMA, usuarios y esquemas sys

Todas las bases de datos contienen dos entidades que son el esquema de información denominado **INFORMATION_SCHEMA** y el usuario **sys**. Son necesarias para el uso interno del servidor y no pueden ser **eliminadas** o **modificadas**.

Usuario guest

Se conoce con este nombre a un usuario "invitado" dentro de cada base de datos. Los permisos del usuario **guest** se aplican a todos aquellos usuarios que tienen acceso a la BD pero que no tienen una cuenta en la base. Si bien no se puede eliminar, si se puede **deshabilitar** revocando el permiso **CONNECT**.

¿Que son los elementos protegibles?

Este concepto hace referencia a aquellos recursos que tienen un acceso regulado por parte del servidor de base de datos mediante un sistema de autorización. Un ejemplo de un elemento protegible sería una **tabla**. Una característica de los elementos protegibles es que pueden **anidarse** unos dentro de otros, por lo tanto es normal encontrar **jerarquías** que son conocidas comúnmente como **ámbitos**. Los tres ámbitos protegibles que se verán a continuación son:

- El servidor
- La base de datos
- El esquema perteneciente a esa base de datos

Ámbito protegible de servidor

Es aquel que contiene a los siguientes elementos:

- Grupo de disponibilidad
- Punto de conexión
- Inicio de sesión
- Rol del servidor
- Base de datos

Ámbito protegible de base de datos

Los elementos protegidos por el alcance de este ámbito son los siguientes:

- Rol de aplicación
- Ensamblado
- La clave asimétrica y asimétrica

- Contrato
- Catálogo de texto completo
- Lista de palabras irrelevantes de texto completo
- Tipo de mensaje
- Enlace de servicio
- Rol de base de datos
- Esquema
- Lista de propiedades de búsqueda
- Usuario

Ámbito protegible de esquema

Por último, en la siguiente lista se muestran los elementos protegidos por un esquema:

- Tipo
- Colección de esquemas XML
- Objetos y subcategorias de objetos (funciones, procedimientos, tablas, vistas, etc)

Control de acceso a un elemento protegible

El acceso a esta clase de elementos se dictamina en base a permisos de **concesión** y **denegación**, principalmente aplicado a **inicios de sesión** y **usuarios** de la base de datos; éstos últimos se denominan entonces como **entidades de seguridad**.

En todos los casos, los permisos a elementos protegibles se gestionan mediante las siguientes sentencias en T-SQL: **GRANT**, **DENY** y **REVOKE**.

Cifrado y certificados en SQL Server

La seguridad de los elementos protegibles en SQL corre a cargo de algo que se conoce como **certificado**, el cual funciona como una "llave" o "tarjeta" de autenticación segura. Contiene una firma digital proporcionada por una entidad de confianza, garantizando al receptor que esa entidad realmente **es quien dice ser**. Por otra parte, el **cifrado** consiste en aplicar **encriptación** a la información que debe ser protegida, y que solo podrá desencriptarse mediante una clave, sea **simétrica o asimétrica** de carácter secreto. Si bien el solo hecho de usar cifrado no vuelve inquebrantable a la instancia de base de datos, si otorga un nivel extra de seguridad ya que incluso en un caso en el que un atacante se haga con información sensible, si no posee las claves necesarias para desencriptarla, la información se vuelve **ilegible e inservible**.

De este tema se desprende otro concepto muy relacionado el cual es la **jerarquía de cifrados de SQL Server**. Básicamente, se trata de un sistema de niveles de cifrado, en el cual cada capa se encargará de proteger a aquella que tiene por debajo; usando combinaciones de certificados, claves **simétricas y asimétricas**. Incluso, estas claves podrían resguardarse de manera **externa** al servidor a fin de aumentar la seguridad.

Consideraciones importantes acerca del uso de cifrado y certificados

- Cifrar los datos usando claves **simétricas** en lugar de certificados y claves **asimétricas** propicia un mejor rendimiento
- La **API** de protección de datos de **Windows** se encarga de cifrar la **clave maestra de servicio** creada por SQL Server durante la instalación, la cual será utilizada para cifrar las **claves maestras de cada base de datos** en la instancia del servidor
- El **Extensible Key Module (EKM)** se encarga de almacenar las claves **fuera** de SQL Server
- Cuando se aplica Cifrado de Datos Transparente (Transparent Data Encryption), se usa una clave **simétrica** que será a su vez protegida por un **certificado protegido** con la **clave maestra** de la base de datos **maestra**, o bien usando una **clave asimétrica** guardada en el **EKM**.
- La característica en común de las claves maestra de servicio y de base de datos es que todas son **claves simétricas**.

A continuación se ofrece una explicación detallada de los distintos **mecanismos de cifrado** nombrados anteriormente para reforzar la comprensión de cada uno de ellos.

Funciones de Transact-SQL

Simplemente consiste en cifrar los elementos en la medida que se **insertan** o **actualizan** en las distintas tablas usando las propias **funciones de T-SQL**.

Certificados, concepto y características

Se entiende como certificado a una **instrucción que contiene una firma digital**, que se ocupa además de enlazar una **clave pública** con una identidad como puede ser una persona física, una empresa, un dispositivo, etc; el cual posee una **clave privada**. Ambas claves están relacionadas matemáticamente. Cualquier certificado válido es emitido por lo que se conoce como una **entidad certificadora**.

En términos generales, un certificado típico contiene la siguiente información:

- la **clave pública** del sujeto
- información de **identificación** de tal sujeto
- el periodo de tiempo en el cual ese certificado tiene **validez**

Todos los certificados tienen **fecha de validez** especificada al momento de su emisión, por lo tanto, una vez **superada esa fecha**, el sujeto del certificado (quien lo usa), deberá solicitar uno **nuevo**.

La gran ventaja que presenta el uso de los certificados es que libera al host de tener que generar contraseñas individuales por cada entidad que se quiere conectar. Es decir, el host solo necesita establecer confianza con la entidad que **emite** los certificados, y por lo tanto asume confianza automáticamente con todos aquellos sujetos que presenten sus certificados firmados ya que se da por sentado que al momento de emitir y firmar un certificado, la entidad emisora ya ha **validado** la **identidad del sujeto**.

Por otra parte, en caso de ser necesario, la entidad emisora puede **revocar** un certificado antes de su fecha de fin de validez, cancelando las obligaciones de la **clave pública** con el **sujeto** al cual se le **adjudica** el certificado.

Claves simétricas y asimétricas

Para comenzar, el concepto de clave simétrica refiere a un tipo de clave de cifrado que se caracteriza por tener la capacidad tanto de **cifrar** como para **descifrar** un bloque de información. Es por esto que, en la práctica, conllevan menos recursos y proporcionan un funcionamiento más rápido por lo cual son la primera opción al trabajar con bases de datos.

Por otra parte, las claves asimétricas son aquellas compuestas por una clave **pública** con su correspondiente clave **privada.** Ambas pueden descifrar aquello que fue cifrado por la otra, y si bien usan más recursos, son una opción más **robusta** para proteger datos.

Cifrado de datos transparente TDE

Se trata de un caso específico de cifrado en el cual se encriptará una base de datos completa usando una clave **simétrica** llamada **clave de cifrado de base de datos**.

Transacciones y bloqueos

Introducción a niveles de aislamiento

En el contexto de SQL Server, y extendido en general a todos los motores de base de datos del mercado, se siguen cuatro principios al momento de realizar **transacciones** y que permiten garantizar no solo su validez sino que también permiten tomar medidas apropiadas en caso de errores del servidor, fallo de energía, inconsistencias en la información resultante, etc. Estos cuatro principios son los siguientes:

- **Atomicidad** → las transacciones son completas, totales, no pueden procesarse a medias
- **Consistencia** → todos los datos que sean guardados deberán ser válidos
- **Aislamiento** → una transacción bajo ningún concepto debe afectar a otras
- **Durabilidad** → los datos que se escriben en la base de datos no pueden perderse

Este principio se lo llama **ACID** por sus siglas en inglés (Atomicity, Consistency, Isolation, Durability).

A continuación se da una explicación de las características de los distintos niveles que se aplican en SQL Server, mostrando además sus posibles desventajas de cara a gestionar transacciones. Estos niveles se toman muy en cuenta en casos de bases de datos muy grandes, en las cuales sucede que se debe garantizar la consistencia de datos cuando muchos usuarios intentan **acceder** o **modificar** los **mismos datos**. En la siguiente lista, se muestran en orden del **menos** restrictivo al **más** restrictivo.

Niveles de aislamiento

READ UNCOMMITED

Este primer nivel conforma aquellas lecturas **no confirmadas**, es decir, que no están "commiteadas". Básicamente, en este nivel de aislamiento lo que sucede es que los usuarios tienen la capacidad de leer datos **aún no confirmados**. Ejemplos de desventajas que se pueden presentar en este nivel son: lecturas **sucias**, lecturas **no repetibles** o incluso **datos fantasma**.

READ COMMITED

En este nivel, aquellos datos que no hayan sido confirmados de antemano **no son leídos**, las lecturas solo se bloquean en caso de escrituras. Presenta problemas como lecturas **no repetibles** y **datos fantasma**. Como detalle extra, este nivel es el que recomienda Microsoft para trabajar con SQL Server.

REPEATABLE READ

Es un nivel que garantiza que dos consultas **SELECT distintas** deben devolver exactamente los **mismos datos**. Puede generar **datos fantasma**.

SERIALIZABLE

Este es el nivel más alto de aislamiento y bloqueo, en el cual las **lecturas** solo se verán afectadas por las **escrituras**.

Nuevo nivel de aislamiento de instantánea

SNAPSHOT Isolation Level

Es una estrategia que permite evitar todos los problemas anteriormente descriptos, sin la necesidad de bloquear las filas, haciendo que una sentencia SELECT siempre devuelva los **mismos datos** cada vez que sea ejecutada.

Características de transacciones implícitas

Se conoce como transacciones explícitas, o más bien, al modo de configuración de SQL Server que **obliga** al motor de base de datos a iniciar **automáticamente** una nueva transacción en cuanto se ejecuta una sentencia de tipo **DML** o **DDL**. En este modo, en tanto una transacción se dé por finalizada, de inmediato **comenzará** la siguiente; o bien si se ejecuta una instrucción que para completarse necesite también de una transacción.

La diferencia a nivel código que implica el uso del modo de transacciones implícitas en SQL Server radica por ejemplo en que **no se usa** la sentencia **BEGIN TRANSACTION** para comenzar la siguiente, por lo tanto se puede afirmar que se crea una **cadena de transacciones**.

Por otra parte, dado que el inicio es automático, el usuario solo se ocupa de hacer la **terminación explícita** de la transacción, ya sea con **COMMIT TRANSACTION** o bien **ROLLBACK TRANSACTION**.

Modo por defecto en SQL Server

Por norma general, las instancias de SQL Server desactivan el modo de transacción implícita en favor del modo **Autocommit**. Esto es así porque el hecho de usar transacciones implícitas implica un riesgo de **bloqueo** y **contención de recursos** conocido como ***locking and blocking***.

Dentro del modo Autocommit, cada sentencia se **interpretará** como una **transacción** y se **confirma automáticamente** al finalizar con éxito; en cambio, si la sentencia fallara, se revierte también de manera automática, lo cual disminuye enormemente el riesgo de dejar **transacciones abiertas**.

En el caso de las transacciones implícitas, solo se requiere el **COMMIT** o **ROLLBACK**, por lo cual se vuelve mucho más fácil **olvidar finalizar la transacción** (es decir, olvidar usar el COMMIT para confirmarla); y en caso que suceda, la transacción queda **abierta** y se mantiene **bloqueando** las tablas que modifica o lee en su ejecución, degradando en gran medida el rendimiento de la base de datos.

Variables tabla en SQL Server

Se caracterizan por ser un tipo de variable especial dentro de T-SQL, con un **ámbito definido** dentro de un SP, función o batch en el cual es declarada.

Producen menos recopilaciones en los Stored Procedures en los cuales se encuentran a comparación de las **tablas temporales**. Además, no requieren de tantos **bloqueos** ni **recursos** como si lo requieren las tablas temporales.

Por otra parte, si se debe notar como principales desventajas en el uso de variables tabla, que una vez definida la variable table, la misma no puede ser **modificada**. Así mismo, no pueden aplicar índices que **no sean agrupados** ni tampoco ser usadas como parte de sentencias **INSERT INTO** o **SELECT INTO**.

A continuación se muestra un ejemplo declaración y asignación de un valor a una variable tabla:

- DECLARE @variableTabla TABLE (campo1 INT, campo2 CHAR(50)); -- declaración
- -- asignación en dos líneas
- INSERT INTO @variableTabla VALUES (1, 'Campo 1');
- INSERT INTO @variableTabla VALUES (2, 'Campo 2');
- SELECT * FROM @variableTabla -- se traen datos

Variables del sistema

Se presenta una categoría especial de variables dentro de SQL Server, se simbolizan usando un **doble signo de @**, por ejemplo **@@TRANSCOUNT**. Solo pueden ser consultadas por el usuario, no se permite su modificación.

Triggers

¿Qué son los Triggers?

Son un tipo de **procedimiento almacenado** especial, que se caracteriza por ser **ejecutado** al momento que se realiza una operación de **CRUD** dentro de una tabla. Dado que se encuentra asociado a una tabla, se invoca automáticamente en el momento que se detecta un cambio en la misma (se "dispara", de allí su nombre). Es por ello que un trigger **no puede ser llamado** de manera manual por el usuario. Por último, se puede notar también que se **identifica** con una **transacción**.

Usos más comunes de los triggers

En general, las formas típicas de usar Triggers engloban las siguientes situaciones:

- Realizar **cambios en cascada** en tablas que se encuentran **relacionadas** dentro de la BD
- Requerir una comprobación o validación más **estricta** que la que puede ofrecer una **constraint** de tipo **CHECK** por ejemplo

- Poder definir mensajes de **error personalizados** para no recaer en mensajes genéricos o poco claros al usuario
- Realizar **mantenimiento rutinario** de datos que por algún motivo no están **normalizados**
- **Comparar** frecuentemente los datos **antes** y **después** de ser insertados

Consideraciones importantes en el uso de Triggers

- Se puede afirmar que los Triggers son **reactivos** (actúan como respuesta a una acción anterior), mientras que las restricciones o constraints son **proactivas** (dado que se comprueban **antes** de interactuar con el dato).
- Dentro de una misma tabla se pueden definir **varios** T**riggers** para modularizar las acciones que se deben llevar a cabo al momento de realizar una operación de CRUD.
- Es importante que tenga **todos** los permisos necesarios para ejecutar todas instrucciones definidas en su **plan de ejecución**.

Categorías de Triggers

Se pueden distinguir dos categorías principales dentro de los Triggers, los cuales son los de tipo **DML** y los **DDL**. A continuación se explican cada uno de ellos y sus correspondientes subtipos.

Triggers DML

Son aquellos Triggers destinados a **manipulación de datos**, de allí su nombre (Data Manipulation Language). Engloba tanto a los Triggers **Instead Of** los cuales se disparan **en lugar de la operación** que los invoca, y a los **After**, que aplican **restricciones** de negocio **avanzadas**.

Triggers DDL

Son los Triggers correspondientes a Data Definition Language. En esta categoría se encontrarán los Triggers de Base de Datos, que garantizan la **integridad** y **consistencia** de datos en las **tablas**; así como también a los Triggers de Servidor, que se disparan automáticamente en **eventos** no a nivel de base de datos sino directamente a **nivel de servidor**.

Creación de un procedimiento Trigger

En el siguiente ejemplo se muestra cómo crear un Trigger sobre una tabla:

- CREATE TRIGGER [nombrePropietario] miTrigger
- ON [nombrePropietario] tablaAsociadaAlTrigger
- [WITH ENCRYPTION]
- {FOR | INSTEAD OF | AFTER}{INSERT, UPDATE, DELETE}
- AS miTrigger;

Restricciones

Dos cuestiones muy importantes a tener en cuenta al momento de trabajar con Triggers son: que si bien la **recursividad** es posible (llamar un Trigger que a su vez en su plan de ejecución llame a otro), **no se recomienda** ya que los niveles de anidación pueden volver muy **complejo** de mantener y depurar. En caso de ser inevitable, SQL Server permite como máximo **32 niveles de anidación**.

Y por otra parte pero no menos importante, en caso que se ejecute una sentencia **TRUNCATE TABLE** los Triggers no son **invocados**.

Modificación y eliminación de un Trigger ya existente

La modificación es un procedimiento muy sencillo, ya que se aplica la misma sentencia necesaria para modificar Stored Procedures convencionales, funciones, tablas, etc; la cual es **ALTER**, seguida del nombre del Trigger que se desea modificar y luego el nuevo plan de ejecución que sobreescribe al anterior.

Y en caso que el Trigger ya no sea necesario, puede ser eliminado de la tabla mediante la sentencia **DROP** seguida del nombre del Trigger.

Consultas dinámicas en SQL Server

Concepto de consultas dinámicas y características más relevantes

Se conoce con este nombre a un tipo de consulta **especial** que se define en tiempo de **ejecución**, y no en tiempo de **compilación** como el resto de consultas que se encuentran comúnmente.

La **primera vez** que se ejecuta una consulta dinámica, la instancia del servidor **determina** el plan de ejecución que deberá seguir esa consulta; luego, dado que ya queda **almacenada** en el sistema, ante cualquier nueva invocación la consulta **usará** el mismo plan de ejecución ya creado. Se muestra a continuación un ejemplo de una consulta dinámica en código SQL:

- DECLARE @tabla NVARCHAR(30), @query NVARCHAR(100);
- SET @tabla = 'Person.Address'; -- tenemos el nombre de la **tabla**
- SET @query = N'SELECT * FROM' + @tabla -- asignamos la query **concatenando**
- EXECUTE(@query) -- esta sentencia **ejecuta** cualquier clase de **código SQL**

Cursores

¿Qué es un cursor en SQL Server?

Un cursor es un objeto que permitirá recorrer de manera **secuencial** una serie de resultados provenientes de una consulta **SELECT**. Es muy útil para acelerar lecturas de resultados de fila en fila, ya que ciertos tipos de resultados se benefician de técnica; permitiendo así operar de manera individual con los registros que conforman el resultado de la consulta original.

Creación e instrucciones de un Cursor

En caso de requerir crear un cursor, el procedimiento es sencillo: mediante la sentencia **DECLARE CURSOR**, en la cual luego se especificará el **comportamiento** que tendrá como por ejemplo el **desplazamiento** usado y la consulta sobre la cual recorrerá los datos.

Lista de sentencias relevantes:

- **OPEN** → se encarga de abrir el cursor del servidor, llenándolo además con la consulta sobre la cual deberá iterar y que fue definida en su creación o asignación posterior.
- **FETCH** → usada para solicitar datos al cursor desde su posición actual, pudiendo recibir además una serie de sub argumentos los cuales son: **NEXT** (devolverá la fila **siguiente** y vuelve a posicionarse en la fila), **PRIOR** (devuelve la fila **anterior** en este caso), luego se encuentra **LAST** (devuelve la **última** fila y la convierte en la fila actual) y **FIRST** (que devuelve la **1er** fila y también pasará a ser la fila actual). Por otra parte, se encuentran las sentencias **ABSOLUTE** {n | @nvar}. Devolverá la posición indicada por **n** en caso que cualquiera de los valores sea **positivo**. Si alguno de ellos es **negativo** devuelve la fila **n antes** de que se finalice el cursor y hace que la fila de vuelta pase a ser la **fila actual**; si fuese 0, **no devuelve filas**. Por último, la sentencia **RELATIVE** es muy similar, solo que devolverá la posición basándose en la posición actual del cursor.

- **CLOSE** → se encarga de **cerrar** el cursor abierto **liberando** los **resultados** y **bloqueos** en sus filas. De igual manera las estructuras de datos generadas quedan **accesibles** para poder usarlas nuevamente más adelante, solo que las actualizaciones y recuperaciones **no se permiten** hasta que el cursor se vuelva a abrir con la sentencia OPEN.
- **DEALLOCATE** → se **quita** la referencia al cursor, si esto sucede, SQL Server **libera** las estructuras de datos que componen el cursor.

En la siguiente página se muestra un ejemplo de código SQL en el cual se aplican todos los conceptos anteriormente explicados en la creación de un cursor.

- DECLARE @descripcion NVARCHAR(100) -- se declara una variable
- -- en la siguiente linea se crea el cursor con su nombre y el **SELECT** asociado
- DECLARE ProdInfo CURSOR FOR SELECT [descripcion]
- FROM Production.ProductDescription
- -- ahora se abre el cursor con la sentencia **OPEN**
- OPEN ProdInfo
- -- con el **FETCH** se guarda el valor de la siguiente fila en la variable
- FETCH NEXT FROM ProdInfo INTO @descripcion
- WHILE @@fetch_status = 0 -– se valida la existencia de la fila leída acá
- BEGIN
- PRINT @descripcion –- se muestra en pantalla con **PRINT**
- -- en la siguiente pasada se repite la asignación
- FETCH NEXT FROM ProdInfo INTO @descripcion
- END
- -– acá se cierra el procedimiento pero quedan sus estructuras disponibles
- **CLOSE** ProdInfo
- **DEALLOCATE** ProdInfo – se finaliza el proceso liberando las estructuras de datos